# JavaScript Prototype and Inheritance

## 1. What is a Prototype?

- Every object in JavaScript has a hidden property [[Prototype]] (accessible via __proto__).
- [[Prototype]] is a reference to another object from which it can inherit properties and methods.
- When a property or method is not found on the object, JavaScript looks for it on the prototype chain.

## 2. Prototype Chain Example

```
const person = {
greet() {
console.log("Hello!");
}
};

const student = Object.create(person);
student.study = function() {
console.log("Studying...");
};

student.greet(); // Hello! (inherited from person)
student.study(); // Studying...
```

Here:
- student.__proto__ === person
- If greet isn’t found in student, JavaScript looks into person.

## 3. Function Prototypes

Functions in JavaScript have a prototype property.

```
function Person(name) {
this.name = name;
}

Person.prototype.sayHi = function() {
console.log("Hi, I’m " + this.name);
};

const p1 = new Person("Alice");
p1.sayHi(); // Hi, I’m Alice
```

- sayHi is defined on Person.prototype and shared across all Person instances.

## 4. Inheritance with Prototypes

Inheritance in prototype works by linking objects.

```
function Animal(name) {
this.name = name;
}
Animal.prototype.speak = function() {
console.log(this.name + " makes a noise.");
};

function Dog(name) {
Animal.call(this, name); // call parent constructor
}
Dog.prototype = Object.create(Animal.prototype); // inherit methods
Dog.prototype.constructor = Dog;

Dog.prototype.speak = function() {
console.log(this.name + " barks.");
};

const d = new Dog("Rex");
d.speak(); // Rex barks.
```

How it works:
- Dog.prototype inherits from Animal.prototype
- Instances of Dog can use both Dog’s methods and Animal’s methods via the prototype chain.

## 5. Prototype Chain Visualization

Example: d = new Dog("Rex")

d ---> Dog.prototype ---> Animal.prototype ---> Object.prototype ---> null

- d has its own properties (name)
- Dog.prototype has Dog methods (speak)
- Animal.prototype has Animal methods (speak if not overridden)
- Object.prototype has default methods like toString.

## Conclusion

- Prototypes are the core mechanism of inheritance in JavaScript.
- The prototype chain enables objects to share behavior without duplication.
- Classes in JavaScript are syntactic sugar over prototypes.